

Full length article

An adaptive defense generation method for mimic honeypots based on Minimax-Q game theory

Shangdong Liu^{a,b,c}, Xukun Qian^{a,b}, Yingrui Cai^{a,b}, Sisi Shao^{b,d}, Wenbo Wang^{a,b}, Fei Wu^{b,c,e}, Yimu Ji^{a,b,c,**}

^a School of Computer Science, Nanjing University of Posts and Telecommunications, Nanjing 210023, China

^b Institute of High Performance Computing and Bigdata, Nanjing University of Posts and Telecommunications, Nanjing 210003, China

^c Jiangsu HPC and Intelligent Processing Engineer Research Center, Nanjing 210003, China

^d School of Cyber Science and Engineering, Wuxi University, Wuxi 214105, China

^e College of Artificial Intelligence, Nanjing University of Posts and Telecommunications, Nanjing 210023, China

ARTICLE INFO

Keywords:

Honeypot

Active defense

Game theory

Minimax-Q algorithm

ABSTRACT

Honeypots serve as powerful tools for protecting critical internal assets by deploying resources without real value as decoys to attract, capture, and analyze attacker behaviors, and have been widely applied in enterprise-level advanced network defense systems. However, as the cybersecurity landscape evolves, attackers are increasingly developing anti-honeypot techniques that can identify and bypass deception mechanisms, or even exploit honeypots as stepping stones to compromise other hosts, thus escalating overall network risks. Consequently, maintaining both strong deception capability and intrinsic security has become a key challenge in network defense. To address this issue, we propose an adaptive defense generation method for mimic honeypots based on Minimax-Q game theory, which effectively enhances both deception and security in honeypot defense systems. Specifically, building upon the multi-level adjudication mechanism, we design mimic honeypots composed of multiple business executors and further define the reward evaluation function of each executor to quantify its defensive effectiveness. Subsequently, an attack–defense game model for mimic honeypots is constructed, and the Minimax-Q reinforcement learning algorithm is employed to solve for the Bayesian Nash equilibrium, enabling the adaptive generation of optimal defense strategies. Furthermore, a dynamic scheduling mechanism is integrated to replace suspicious executors in real time, thereby preventing attackers from further compromising other hosts. Finally, experimental results show that compared with existing methods, the proposed mimic honeypot demonstrates significant advantages in both deception capability and intrinsic security, offering new insights for building more robust network defense systems.

1. Introduction

With the rapid advancement of informatization and digitalization, cybersecurity has become a focal global concern (Perwej et al., 2021). In recent years, cyber incidents have escalated sharply, with attackers frequently exploiting vulnerabilities spanning hardware, software, and communication layers (Safaei Pour et al., 2023). According to the 2024 U.S. National Vulnerability Database (NVD), the number of CVE vulnerabilities increased by 38% year-over-year, underscoring the growing severity of risks across critical infrastructure, enterprise systems, and common applications (Aslan et al., 2023; Zambianco et al., 2024). Although patching remains the dominant mitigation approach, enterprises often delay updates due to stability and compatibility concerns,

leaving vulnerabilities exposed for extended periods (Siwakoti et al., 2023; Wu et al., 2023). Moreover, as attack techniques evolve, cyber confrontation has shifted from point-to-point engagements to continuous, domain-wide dynamic games. In highly homogeneous operating-system and application environments, a single vulnerability can be exploited at scale, creating severe threats (Chernikova et al., 2023). Traditional passive defenses (e.g., firewalls, intrusion detection systems) are constrained by their posterior and specificity-driven nature (Zheng et al., 2022), making them ineffective against complex, adaptive attacks and enabling adversaries to bypass boundaries with relative ease (Tian et al., 2021; Geng et al., 2024). Against this backdrop, proactive defense technologies—aiming to “change the rules of the

* Corresponding author at: School of Cyber Science and Engineering, Wuxi University, Wuxi 214105, China.

** Corresponding author at: School of Computer Science, Nanjing University of Posts and Telecommunications, Nanjing 210023, China.

E-mail addresses: shaosisi@cwuxu.edu.cn (S. Shao), jiym@njupt.edu.cn (Y. Ji).

game”—have emerged. Their core idea is to intervene early in the attack chain through proactive sensing, dynamic adjustment, deception, inducement, and countermeasures, forcing attackers to reveal intentions and techniques, and shifting defenders from passive to active (Rehman et al., 2024; Kelly et al., 2021).

As a representative proactive defense technology, honeypots have been widely deployed for threat intelligence collection and attack attribution due to their ability to mimic real systems, lure attackers with decoy resources, and record malicious behavior (Zhu et al., 2024). Their effectiveness derives from presenting adversaries with seemingly genuine yet valueless system resources that reveal attack paths and provide early warning without affecting critical services. However, with the evolution of anti-honeypot techniques, adversarial methods have become increasingly sophisticated, reducing honeypot effectiveness in real scenarios (Li et al., 2025; Mohammadzad and Karimpour, 2023). The rising prevalence of security bypass vulnerabilities further illustrates attackers' enhanced ability to evade existing defenses (Touré et al., 2024). More critically, some adversaries can exploit latent vulnerabilities within virtual honeypots to conduct reverse penetration, posing significant risks to internal networks (Shao et al., 2026).

Cyberspace Mimic Defense (CMD) offers a feasible solution to such intrinsic risks (Hu et al., 2018). Inspired by biological immunology (Wu, 2022), CMD highlights the principle that “architecture determines security”. Based on Dissimilar Redundancy Structures (DRS), it introduces Dynamic Heterogeneous Redundancy (DHR), which constructs multiple implementation-diverse yet functionally equivalent executors and employs dynamic scheduling and multimodal adjudication for real-time attack sensing and proactive response. Unlike traditional static defenses, DHR introduces uncertainty and heterogeneity at the deployment level, disrupting adversaries' prior knowledge, mitigating single-path exploits, and enhancing resilience against advanced persistent threats (Shao et al., 2023; Liu et al., 2024).

Existing research on honeypots mainly enhances deception and threat intelligence collection, yet systematic mechanisms for intrinsic honeypot security and virtual-escape resistance remain limited (Ilg et al., 2023). Current strategy-generation methods also struggle to balance long-term rewards with immediate defensive effectiveness under dynamic adversarial environments (Shao et al., 2025). To address this gap, we integrate honeypot technology with mimic defense. Specifically, without compromising trapping effectiveness, potentially compromised service components are dynamically replaced, reducing risks of virtual escape and reverse penetration. Nevertheless, the straightforward combination of mimic defense and honeypots still faces limitations: it lacks quantitative evaluation of defense benefits versus resource overhead, and it does not fully capture the game-theoretic nature of cyber confrontations, leaving strategies vulnerable to adversarial probing.

To tackle these challenges, this paper proposes an adaptive defense generation method for mimic honeypots based on Minimax-Q game playing. The method leverages mimic defense concepts by arranging heterogeneous executors from both service and virtualization layers into a defense action set. On this basis, we construct a mimic honeypot attack-defense game model, solve for the Bayesian Nash equilibrium using the Minimax-Q multi-agent reinforcement learning algorithm, and generate adaptive defense strategies accordingly. The proposed mimic honeypot replaces suspicious executors via multi-level adjudication and dynamic scheduling mechanisms, thereby preventing further host compromise, improving deception effectiveness, and significantly strengthening intrinsic honeypot security.

- We design an adaptive defense generation architecture for mimic honeypots. Based on the Bayesian Nash equilibrium derived from game-theoretic analysis, the architecture dynamically adjusts honeypot defense strategies. In addition, a dynamic scheduling mechanism at the virtualization layer is employed to securely replace business executors suspected of virtual escape, providing a new implementation path for enhancing the intrinsic security of honeypots.

- We construct an attack-defense model for mimic honeypots based on incomplete-information stochastic games, define a reward evaluation function for business executors, and solve the Bayesian Nash equilibrium using the Minimax-Q multi-agent reinforcement learning algorithm. This enables the dynamic and adaptive generation of optimal defense strategies, thereby enhancing both deception and protection capabilities of the honeypot while making efficient use of network resources.
- We implement a mimic honeypot simulation environment for web service systems. The proposed method is validated through game-theoretic analysis of equilibrium correctness, as well as security testing that demonstrates the effectiveness of executor scheduling and resilience against virtual escape attacks.

2. Related work

In contemporary cyber offense-defense confrontations, honeypot technology has become a crucial proactive defense mechanism. By deploying deceptive virtual resources, honeypots attract adversaries and collect behavioral information for subsequent analysis and mitigation. However, traditional honeypots suffer from fixed deployment, limited functionality, and insufficient situational awareness, making them prone to exposure. With the rapid advancement of honeypot-detection and anti-honeypot techniques, evading deception—or even exploiting honeypot vulnerabilities for reverse penetration—has become a key adversarial strategy. Attackers may leverage compromised honeypots as stepping stones to launch large-scale internal attacks, posing serious threats to defense systems (Javadpour et al., 2024). Consequently, achieving strong deception while ensuring intrinsic honeypot security has become an important research direction. Advances in layered architectures and virtualization have reduced identifiability, while game-theoretic and machine-learning methods provide optimization for dynamic deployment and long-term adaptability.

From an architectural and deployment perspective, researchers have explored virtualization and layered strategies to enhance honeypot stealthiness, scalability, and flexibility. Franco et al. (2021) systematically reviewed virtual honeypots in IoT and IIoT, emphasizing their low cost, scalability, and ability to emulate diverse protocols. Zarca et al. (2020) proposed an SDN- and NFV-based virtual IoT honeynet with autonomous, policy-driven orchestration to support heterogeneous attack scenarios. Shivaprasad et al. (2024) developed a multi-layered honeypot architecture integrating traffic analyzers into a seven-layer defense system, achieving end-to-end coverage from anomaly detection to proactive defense. While these works enhance flexibility and stealth, they provide limited mechanisms for mitigating threats such as virtual escape or reverse exploitation.

In strategy optimization, game theory has become a key tool for modeling dynamic honeypot deployment and interaction. Pawlick et al. (2019) developed a signaling-game-based deception model, revealing the defensive importance of high-accuracy detectors. Anwar et al. (2021) extended honeypot allocation on attack graphs from static Nash equilibrium to dynamic game analysis, while Panda et al. (2022) presented HoneyCar, which employs repeated incomplete-information games and vulnerability assessment data to optimize deployment. Anwar et al. (2022) proposed a two-stage adaptive allocation framework combining active games with Monte Carlo tree search, improving adaptability under uncertain environments. Sayed et al. (2023) introduced the probabilistic Course-of-Action Stackelberg Game (CoASG), constructing attack graphs and reward matrices to derive optimal strategies for early attack-pattern detection. These studies provide a theoretical foundation for honeypot decision-making, but often rely on simplified assumptions and lack mechanisms to balance defensive reward and resource overhead. As an emerging security concept, Cyberspace Mimic Defense (CMD) generates advantageous asymmetric uncertainty for defenders. Chen et al. (2021) applied incomplete-information dynamic games and Markov processes to optimal mimic

defense, highlighting the key role of dynamic games in structured defense. Building on this, researchers have integrated game theory and mimic defense into honeypots to address intrinsic security and active defense challenges. Specifically, [Shao et al. \(2026\)](#) proposed a secure deployment method for mimic honeynets by combining game-theoretic equilibrium solving with dynamic heterogeneous redundancy, effectively resolving intrinsic security and optimal deployment issues in complex adversarial environments.

From the perspective of intelligence and adaptability, machine learning and reinforcement learning have been increasingly applied to improve honeypot dynamism. [Wang et al. \(2024\)](#) integrated evolutionary game theory with Q-learning to optimize dynamic service-switching strategies. [Commeey et al. \(2025\)](#) developed the BHICS system, which uses lightweight ML to dynamically convert IoT nodes into honeypots and leverages blockchain for event recording. [He et al. \(2024\)](#) proposed a deception asset selection algorithm based on deep reinforcement learning and differential games, which significantly improves the convergence efficiency of defense strategies. [Naik et al. \(2021\)](#) introduced D-FRI-Honeypot, which employs fuzzy rule interpolation for adaptive fingerprinting resistance. [Selva Kumar et al. \(2025\)](#) presented ADHDN, a distributed honeypot detection network combining DGAN and DHMM for high-interaction environments. [Veluchamy and Kathavarayan \(2022\)](#) developed DARLH, integrating deep adaptive reinforcement learning with DRNNs for bidirectional DoS monitoring. These approaches enhance honeypot adaptability, but challenges remain in interpretability, computational cost, and training stability under adversarial conditions.

In summary, existing research has significantly advanced honeypot deception, flexibility, and intelligence. Nevertheless, several gaps persist: (1) insufficient attention to intrinsic honeypot security and virtual-escape resistance; (2) overly idealized game-theoretic assumptions that cannot fully reflect real adversarial complexity; and (3) lack of unified evaluation frameworks and mechanisms to balance defensive gains against resource consumption. Therefore, there is an urgent need for a honeypot defense mechanism that integrates strong deception with intrinsic security, combines game-theoretic equilibria with reinforcement-learning-based adaptive optimization, and ensures both effective trapping and resilience against reverse exploitation, thereby supporting long-term intelligent defensive stability.

3. Adaptive defense generation architecture for mimic honeypots

The adaptive defense generation method for mimic honeypots based on Minimax-Q game theory arranges heterogeneous business executors, consisting of both service and virtualization layers, into executor groups, which form a set of defensive actions. An attack–defense game model is then constructed, and the Bayesian Nash equilibrium is solved using the Minimax-Q reinforcement learning algorithm. According to the equilibrium results, the executor group with the highest reward under the current attack is selected, and a mimic honeypot is adaptively generated for defense. The mimic honeypot replaces suspicious business executors (i.e., executors suspected of compromise) via a multi-level adjudication mechanism and a dynamic scheduling algorithm, thereby preventing attackers from intruding into other hosts. Taking web services as an example, the adaptive defense generation architecture for mimic honeypots is shown in [Fig. 1](#).

Business Executors. The business executor module consists of multiple executors, each composed of a service implementation layer and a virtualization layer. The service implementation layer encompasses various component sets required to realize web services, such as scripting languages, web servers, and operating systems. Each component set further contains multiple functionally equivalent but structurally diverse components; for example, the operating system set may include CentOS, Windows, and Ubuntu. By arranging and combining these business executors, executor groups are formed. When the redundancy level of the mimic architecture is set to three, the system's security

requirements can be satisfied. Accordingly, in this study, each executor group is configured to contain three business executors, resulting in C_s^3 executor groups in total.

Mimic Honeypot. According to the characteristics of honeypot attack–defense scenarios, an incomplete-information stochastic game model is constructed, and a Markov decision process is used to describe the state transitions of attacks and defenses. The Bayesian Nash equilibrium of this game model is solved using the Minimax-Q reinforcement learning algorithm. Based on the equilibrium results, the optimal executor group is selected to construct the mimic honeypot, which then guides network defense decision-making. Inside the mimic honeypot, a multi-level adjudication mechanism identifies anomalies at the virtualization layer and reports them to the dynamic scheduling module, while the service layer provides interactive outputs to attract attackers for deeper engagement.

Dynamic Scheduling. The dynamic scheduling module replaces and sanitizes business executors that exhibit anomalies in the virtualization layer. Based on the current outputs of the mimic honeypot executors, it matches executor groups that contain only uncompromised executors and selects the one with the highest defensive capability for replacement.

4. Adaptive defense generation method for mimic honeypots based on minimax-Q game theory

An incomplete-information stochastic game is used to model and analyze the attack–defense interactions of mimic honeypots, where state transitions are represented by a Markov decision process (MDP). In practice, mimic honeypot networks are highly dynamic, and the transition probabilities of network states vary over time. As a result, the classical state–action value function Q in stochastic games cannot be directly applied to compute the Bayesian Nash equilibrium. To overcome this limitation, we incorporate reinforcement learning techniques and adopt the Minimax-Q multi-agent reinforcement learning algorithm to derive optimal game strategies. This approach enables the selection of the executor group with the highest reward under the current attack, thereby supporting the adaptive generation of mimic honeypots for defense.

4.1. Attack–defense game model for mimic honeypots

Before building the model, we make the following assumptions regarding the attacker and the mimic honeypot:

- (1) Both attacker and defender (i.e., the attacker and the mimic honeypot) are rational agents in an adversarial relationship, with no possibility of collusion. The gain of one side implies the loss of the other, such that the attacker's reward and the defender's reward sum to zero.
- (2) During the attack–defense process, the mimic honeypot cannot precisely identify the type of attacker, but a prior probability distribution of attacker types is assumed.

The attack–defense process of the mimic honeypot is discretized into a sequence of time slots, each corresponding to a network state. In each time slot t , the attacker and defender select their respective actions based on the current state s . Under the joint influence of these actions, the state transitions to a new state s' . The state transition relationships across time slots are illustrated in [Fig. 2](#). Moreover, the actions taken by both parties in time slot t are observable to the opponent in the subsequent time slot $t + 1$.

Based on the above assumptions, we construct an attack–defense game model for mimic honeypots, which is defined as an eight-tuple $G = \{N, S, A, H, \pi, U, \Phi, P_A\}$. Its components are described as follows:

- (1) $N = \{\text{attacker, honeypot}\}$ denotes the set of game players, representing the attacker and the mimic honeypot.

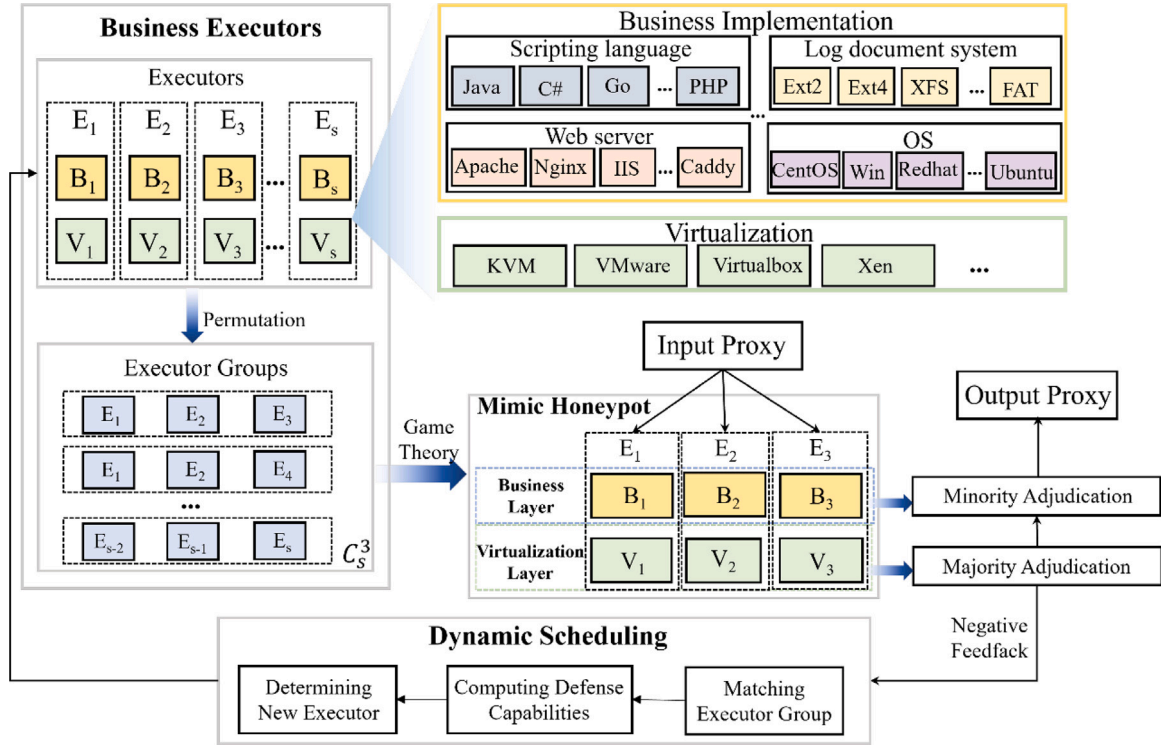


Fig. 1. Adaptive defense decision architecture of mimic honeypots.

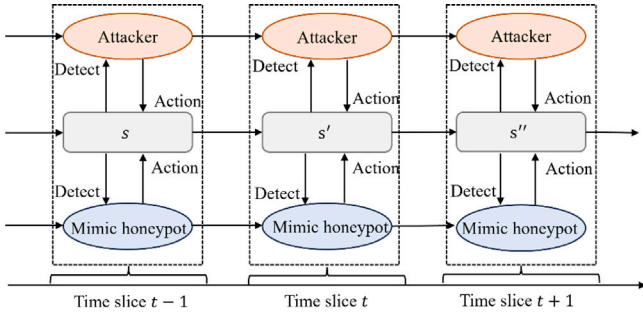


Fig. 2. The attack-defense process of mimic honeypots.

- (2) $S = \{s_1, s_2, \dots, s_n\}$ denotes the set of all attack-defense states in the game, where each state corresponds to a network condition.
- (3) $A = \{A_1, A_2, \dots, A_n\}$ denotes the attacker's action sets, where for each state s_k , the action set is $A_k = \{a_1, a_2, \dots, a_m\}$, representing the possible actions of the attacker in that state.
- (4) $H = \{H_1, H_2, \dots, H_n\}$ denotes the action sets of the mimic honeypot, where for each state s_k , the action set is $H_k = \{h_1, h_2, \dots, h_m\}$, representing the selection of executor groups used to construct the mimic honeypot.
- (5) $\pi = \{\pi_a(s_k, \varphi_i), \pi_h(s_k)\}$ denotes the strategy sets of the attacker and the mimic honeypot. Specifically, $\pi_a(s_k, \varphi_i) = (\sigma_a(s_k, a_1, \varphi_i), \dots, \sigma_a(s_k, a_m, \varphi_i))$ represents the strategy of an attacker of type φ_i in state s_k , while $\pi_h(s_k) = (\sigma_h(s_k, h_1), \dots, \sigma_h(s_k, h_m))$ represents the strategy of the mimic honeypot in state s_k . The term $\sigma_a(s_k, a_m, \varphi_i)$ denotes the probability that an attacker of type φ_i selects action a_m in state s_k . Similarly, $\sigma_h(s_k, h_m)$ denotes the probability that the mimic honeypot selects executor group h_m in state s_k .
- (6) $U = \{Q, V\}$ denotes the utility function. Here, $Q = (Q_a, Q_h)$ denotes the state-action value functions of the attacker and the

mimic honeypot. For example, $Q_h(s_i, a, d, \varphi_j)$ denotes the state-action value of the mimic honeypot in state s_i when an attacker of type φ_j selects action a and the defender selects action d . $V = (V_a, V_h)$ denotes the state value functions of the attacker and the mimic honeypot, where $V_h(s_i, \pi_a(s_i, \varphi_j), \pi_h(s_i), \varphi_j)$ denotes the value of the mimic honeypot under attacker type φ_j in state s_i .

- (7) $\Phi = \{\varphi_i, i = 1, 2, \dots, n\}$ denotes the set of attacker types.
- (8) $P_A = \{p_A(s_i, \varphi_1), p_A(s_i, \varphi_2), \dots, p_A(s_i, \varphi_n)\}$ denotes the prior probability distribution of attacker types for mimic honeypot in state s_i .

In the defined model, the strategies π are represented in probabilistic form, specifying the actions chosen by the attacker and the mimic honeypot in each network state. The outcomes of these strategic interactions not only determine the immediate reward but also affect the future returns. Therefore, the value functions include both immediate rewards and discounted future returns. The calculations of Q_h and V_h are given as follows:

$$V_h(s_i, \pi_a(s_i, \varphi_j), \pi_h(s_i), \varphi_j) = \sum_{a \in A_i} \sigma_a(s_i, a, \varphi_j) \sum_{d \in D_i} \sigma_d(s_i, d) Q_h(s_i, a, d, \varphi_j), \quad (1)$$

$$Q_h(s_i, a, d, \varphi_j) = R_h(s_i, a, d, \varphi_j) + \gamma \sum_{s' \in S} p(s' | s_i, a, d) V_h(s', \pi_a^*(s', \varphi_j), \pi_d^*(s'), \varphi_j), \quad (2)$$

where γ is the discount factor, with values in the range $0 < \gamma \leq 1$, determining the relative importance of future rewards. $p(s' | s_i, a, d)$ denotes the state transition probability. $R_h(s_i, a, d, \varphi_j)$ represents the immediate reward of the mimic honeypot in state s_i , which is determined by deception capability, security defense capability, and computational overhead. The specific calculation of these factors is provided in Section 5.2.

4.2. Mimic honeypot reward evaluation

A comprehensive evaluation of the reward of a mimic honeypot must take into account both its characteristics and functional roles. This evaluation is a critical step in the game process and serves as an important basis for defense decision-making. Unlike other threat-blocking defense techniques, a mimic honeypot aims to lure attackers into a controllable environment while ensuring its own safety. Therefore, the reward function must guarantee the normal operation of the mimic honeypot while mitigating the damage caused by attacks. The reward is defined as follows:

$$R_h(s_i, a, d, \varphi_j) = DV * SDV * \frac{1}{Overhead}, \quad (3)$$

where DV denotes the deception value of the mimic honeypot, SDV denotes its security defense value, and $Overhead$ refers to the computational resources consumed. DV and SDV represent two complementary utility dimensions rather than probabilistic events; therefore, their multiplicative aggregation does not rely on an assumption of statistical independence. The unit exponents indicate that the components have equal proportional sensitivity. The contributions and sensitivity of the individual components are further evaluated in Section 5.2. Among these, DV and SDV are positively correlated with the reward, while $Overhead$ is negatively correlated. The quantification of these three indicators is introduced below.

(1) Deception Value

The deception value DV of a mimic honeypot is determined by the business layer similarity $Similarity(B)$ and sweetness $Sweetness$, calculated as

$$DV = Similarity(B) * Sweetness. \quad (4)$$

$Similarity(B)$ is determined by the distribution of internal vulnerabilities in the honeypot's business implementation. At the business layer, we expect attackers to actively expose more vulnerabilities, thereby attracting them into the honeypot. Therefore, the greater the similarity $Similarity(B)$ of the business layer, the stronger the deception capability of the honeypot. We assume that the set of business-layer vulnerabilities contained in the mimic honeypot is $V_B = \{v_1, v_2, v_3, \dots, v_w\}$. The vulnerability vector of business implementation h in the mimic honeypot is expressed as $\mathbf{v}_{B,h} = [v_{h1}, v_{h2}, v_{h3}, \dots, v_{hj}, \dots, v_{hw}]^T$. When vulnerability v_j exists, $v_{hj} = 1$; otherwise, $v_{hj} = 0$. The vulnerability vector of business implementation x is expressed as $\mathbf{v}_{B,x} = [v_{x1}, v_{x2}, v_{x3}, \dots, v_{xj}, \dots, v_{xw}]^T$. The Jaccard similarity coefficient is adopted to measure the similarity between the vulnerability distributions of two business implementations. Then, the similarity between $\mathbf{v}_{B,h}$ and $\mathbf{v}_{B,x}$ is defined as

$$J_B(\mathbf{v}_{B,h}, \mathbf{v}_{B,x}) = \frac{|\mathbf{v}_{B,h} \cap \mathbf{v}_{B,x}|}{|\mathbf{v}_{B,h} \cup \mathbf{v}_{B,x}|}, \quad (5)$$

where $|\mathbf{v}_{B,h} \cap \mathbf{v}_{B,x}|$ denotes the number of components in which the values of the vector $\mathbf{v}_{B,h}$ and $\mathbf{v}_{B,x}$ are equal to 2, and $|\mathbf{v}_{B,h} \cup \mathbf{v}_{B,x}|$ denotes the number of components in which the values of the vector $\mathbf{v}_{B,h}$ and $\mathbf{v}_{B,x}$ are greater than 0. Since each executor group consists of three executors, let their business-layer vulnerability vectors be $\mathbf{v}_{B,1}$, $\mathbf{v}_{B,2}$ and $\mathbf{v}_{B,3}$. The business-layer similarity is calculated as the arithmetic mean of the three pairwise Jaccard similarity coefficients:

$$Similarity(B) = \frac{J(\mathbf{v}_{B,1}, \mathbf{v}_{B,2}) + J(\mathbf{v}_{B,1}, \mathbf{v}_{B,3}) + J(\mathbf{v}_{B,2}, \mathbf{v}_{B,3})}{3}. \quad (6)$$

The $Sweetness$ is determined by the Common Vulnerability Scoring System (CVSS) of internal vulnerabilities and their exploitation frequency by attackers. We assume that the sweetness of a single vulnerability is defined as

$$Sweetness_j = CVSS_j \times f_j, \quad (7)$$

where $CVSS_j$ denotes the severity of the damage caused by vulnerability j , and f_j denotes its exploitation frequency. Specifically, $f_j = A_j / A$, where A_j represents the number of attacks against vulnerability j captured by the honeypot within a given time period, and A represents the total number of attacks encountered in the same period. Thus, the sweetness of a single business implementation is expressed as

$$Sweetness_b = \sum_j CVSS_j \times f_j, \quad (8)$$

where v denotes the total number of vulnerabilities contained in the business implementation. Accordingly, the sweetness of a single mimic honeypot is

$$Sweetness = \sum_i \sum_j CVSS_{ij} \times f_{ij}, \quad (9)$$

where n denotes the number of business implementations in a single mimic honeypot.

(2) Security Defense Value

The security defense value SDV of a mimic honeypot is determined by the virtualization layer diversity $Diversity(V)$ and confidence $Confidence$, calculated as

$$SDV = Diversity(V) * Confidence. \quad (10)$$

Unlike the business layer, the virtualization layer aims to reduce common vulnerabilities among different virtualization technologies and prevent an attacker from exploiting the same vulnerability to simultaneously compromise multiple executors. This design reduces the risks of common-mode vulnerability exploitation and virtualization escape attacks. Therefore, the Jaccard distance is adopted to measure the heterogeneity of vulnerability distributions among different virtualization technologies. We assume that the set of virtualization-layer vulnerabilities contained in the mimic honeypot is $U_V = \{u_1, u_2, u_3, \dots, u_m\}$. The virtualization-layer vulnerability vector of executor h is expressed as $\mathbf{u}_{V,h} = [u_{h1}, u_{h2}, u_{h3}, \dots, u_{hj}, \dots, u_{hm}]^T$. When vulnerability u_j exists, $u_{hj} = 1$; otherwise, $u_{hj} = 0$. Similarly, the virtualization-layer vulnerability vectors of executors h and x are denoted by $\mathbf{u}_{V,h}$ and $\mathbf{u}_{V,x}$, respectively. The Jaccard distance between two virtualization technologies is defined as

$$d(\mathbf{u}_{V,h}, \mathbf{u}_{V,x}) = 1 - \frac{|\mathbf{u}_{V,h} \cap \mathbf{u}_{V,x}|}{|\mathbf{u}_{V,h} \cup \mathbf{u}_{V,x}|}, \quad (11)$$

where $|\mathbf{u}_{V,h} \cap \mathbf{u}_{V,x}|$ denotes the number of components in which the values of the vector $\mathbf{u}_{V,h}$ and $\mathbf{u}_{V,x}$ are equal to 2, and $|\mathbf{u}_{V,h} \cup \mathbf{u}_{V,x}|$ denotes the number of components in which the values of the vector $\mathbf{u}_{V,h}$ and $\mathbf{u}_{V,x}$ are greater than 0. Since each executor group consists of three executors, let their virtualization-layer vulnerability vectors be $\mathbf{u}_{V,1}$, $\mathbf{u}_{V,2}$ and $\mathbf{u}_{V,3}$. The virtualization-layer diversity is calculated as the arithmetic mean of the three pairwise Jaccard distances:

$$Diversity(V) = \frac{d(\mathbf{u}_{V,1}, \mathbf{u}_{V,2}) + d(\mathbf{u}_{V,1}, \mathbf{u}_{V,3}) + d(\mathbf{u}_{V,2}, \mathbf{u}_{V,3})}{3}, \quad (12)$$

In the initial state, all virtualization techniques in the virtualization layer are considered equally credible. During the operation of the mimic honeypot, as the system runs stably over time, the confidence gradually increases. If an anomaly occurs in a virtualization technique, its confidence will decrease significantly. When the confidence drops below the threshold δ , the virtualization technique will be discarded and replaced.

Let $y_i(t)$ denote the local decision result of virtualization technique i at time t , $Y(t)$ denote the global decision result output by the majority adjudicator of the virtualization layer at time t , and $\zeta_i(T)$ denote the number of times virtualization technique i has been selected into the

honeypot up to time T . Therefore, the $Confidence_i(T)$ of virtualization technique i by time T is defined as

$$Confidence_i(T) = \frac{\sum_{t \in \zeta_i(T)} f(|Y(t) - y_i(t)|)}{|\zeta_i(T)|}, \quad (13)$$

where $f(\cdot)$ is defined as $f(x) = \begin{cases} 1, & x = 0 \\ 0, & x \neq 0 \end{cases}$. At time t , if virtualization technique i is not selected, its confidence remains the same as in the previous time step. If it is selected, then its confidence is dynamically updated as follows:

$$Confidence_i(t) = \begin{cases} \frac{|\zeta_i(t-1)|h_i(t-1) + f(|Y(t) - y_i(t)|)}{|\zeta_i(t-1)| + 1}, & i \in H \\ C_i(t-1), & i \notin H, \end{cases} \quad (14)$$

where $Confidence_i(0)$ denotes the initial confidence of service entity i , which is set to 0.5. This value is adopted as a neutral prior, representing neither prior trust nor prior distrust. Since the same initial value is assigned to all virtualization methods, it does not introduce an initial bias among candidate executor groups. The sensitivity of this setting is further evaluated in Section 5.2. Let $Confidence(t) = [Confidence_1(t), Confidence_2(t), \dots, Confidence_n(t)]$ denote the confidence levels of the n virtualization methods within the honeypot in time slot t .

(3) Overhead

Constructing a mimic honeypot to attract attackers inevitably incurs computational resource *Overhead*. Different business implementations or virtualization techniques require different amounts of resources to process the same task, thereby leading to varying levels of overhead. If computational overhead is ignored, the performance of the real system may be degraded, further affecting normal services. The specific calculation is as follows:

$$Overhead_k = CPU_k + Memory_k, \quad (15)$$

where CPU_k denotes the CPU overhead required by executor group k , and $Memory_k$ denotes the memory overhead required by executor group k .

4.3. Bayesian Nash equilibrium solving based on minimax-Q algorithm

The Bayesian Nash Equilibrium seeks the optimal strategies for both parties in the game, where neither party can further improve its reward by unilaterally adjusting its own strategy. In a game state s_i for all attack strategies $\pi_a(s_i, \varphi_j)$ under each attack type φ_j , we have

$$V_a(s_i, \pi_a^*(s_i, \varphi_j), \pi_d^*(s_i), \varphi_j) \geq V_a(s_i, \pi_a(s_i, \varphi_j), \pi_d^*(s_i), \varphi_j). \quad (16)$$

For all defense strategies $\pi_d(s_i)$, we have

$$\begin{aligned} & \sum_{j=1}^n p_A(s_i, \varphi_j) V_d(s_i, \pi_a^*(s_i, \varphi_j), \pi_d^*(s_i), \varphi_j) \\ & \geq \sum_{j=1}^n p_A(s_i, \varphi_j) V_d(s_i, \pi_a^*(s_i, \varphi_j), \pi_d(s_i), \varphi_j). \end{aligned} \quad (17)$$

Thus, the strategy $\{\pi_a^*(s_i, \varphi_1), \dots, \pi_a^*(s_i, \varphi_n), \pi_d^*(s_i)\}$ constitutes a Bayesian Nash Equilibrium in the network state s_i . In the constructed mimic honeypot attack-defense game model, the network state is influenced not only by the actions of the attacker and the mimic honeypot but also by other complex factors in the network. This leads to unknown state transition probabilities, and the reward function Q also varies accordingly. Therefore, solving for the Bayesian Nash Equilibrium in the mimic honeypot attack-defense game model is highly complex. To address this, we introduce the Minimax-Q reinforcement learning algorithm to simulate the behaviors of both sides, allowing strategies to be iteratively adjusted and gradually approach equilibrium. The Minimax-Q algorithm formulates a linear programming problem based

Algorithm 1 Bayesian Nash Equilibrium Solving Based on the Minimax-Q Algorithm

1. Initialize the Q-table $Q_h(s_i, a, d, \varphi_j)$ and the value function $V_h(s_i, \pi_a(s_i, \varphi_j), \pi_h(s_i), \varphi_j)$ for the current network state s_i .
2. Initialize the learning rate α and discount factor γ .
3. The mimic honeypot selects a defense action d according to the current network state s_i .
4. Obtain the next network state s'_i , calculate the immediate reward $R_h(s_i, a, d, \varphi_j)$ of the mimic honeypot, and observe the attack strategy a adopted by the attacker in state s_i .
5. Update $Q_h(s_i, a, d, \varphi_j)$ according to Eq. (19).
6. Update $V_h(s_i, \pi_a(s_i, \varphi_j), \pi_h(s_i), \varphi_j)$ according to Eq. (18).
7. In state s'_i , perform ϵ -greedy exploration ($0 \leq \epsilon \leq 1$): with probability ϵ , randomly select the next action; with probability $1 - \epsilon$, select the current optimal action based on existing policies.
8. Repeat steps 3–7 until the algorithm reaches the predefined maximum number of iterations, and output the optimal executor group for the mimic honeypot.

on the minimax principle to solve for the Nash equilibrium strategy in each state, and then applies the temporal-difference (TD) method from Q-learning to iteratively update the state value function or state-action reward function.

We map the agent in Minimax-Q to the mimic honeypot in the attack-defense game model. The agent's actions correspond to the defense actions of the mimic honeypot, i.e., the selection of executor groups. The agent's immediate reward corresponds to the immediate reward of the mimic honeypot in the game model. In the stochastic game between the two agents, given a state s_i , the state value function of the agent is expressed as

$$\begin{aligned} & V_h(s_i, \pi_a^*(s_i, \varphi_j), \pi_d^*(s_i), \varphi_j) \\ & = \max_{\pi_h(s_i, \cdot)} \min_{a_{-i} \in A_{-i}} \sum_{d_j \in D_h} Q_h(s_i, a_{-i}, d_j, \varphi_j) \pi_h(s_i, d_j), \end{aligned} \quad (18)$$

where $-i$ denotes the opponent of agent i , i.e., the attacker, and $Q_h(s_i, a_{-i}, d_j, \varphi_j)$ represents the state-action value function. If $Q_h(s_i, \cdot)$ were known, the Nash equilibrium of state s_i could be solved directly using linear programming. However, in multi-agent reinforcement learning, $Q_h(s_i, \cdot)$ is unknown. Therefore, the temporal-difference method from Q-learning is introduced to approximate the equilibrium point, and the update rule is defined as

$$\begin{aligned} Q_h(s_i, a, d, \varphi_j) &= Q_h(s_i, a, d, \varphi_j) + \alpha (R_h(s_i, a, d, \varphi_j) \\ &+ \gamma V_h(s_i, \pi_a(s_i, \varphi_j), \pi_h(s_i), \varphi_j) - Q_h(s_i, a, d, \varphi_j)) \end{aligned} \quad (19)$$

where $R_h(s_i, a, d, \varphi_j)$ denotes the immediate reward, and α is the learning rate ($0 < \alpha \leq 1$), which determines the degree of influence of each update on the agent. Therefore, the Bayesian Nash Equilibrium solving algorithm based on the Minimax-Q algorithm is summarized in Algorithm 1.

4.4. Adaptive defense generation for mimic honeypots

Unlike other threat-blocking defense techniques, the purpose of a mimic honeypot is to lure attackers into performing attacks within a controllable environment inside the honeypot. When the honeypot exhibits anomalous outputs, we must determine, case by case, whether the anomaly endangers the honeypot itself. Therefore, we partition the executor groups within a mimic honeypot into a virtualization layer and a business layer. The virtualization layer hosts the corresponding virtualization technologies for the mimic honeypot and uses a majority adjudication to mask attacks on the honeypot core that exploit the virtualization layer. The business layer simulates real service hosts by providing web service functionality to visitors and uses minority

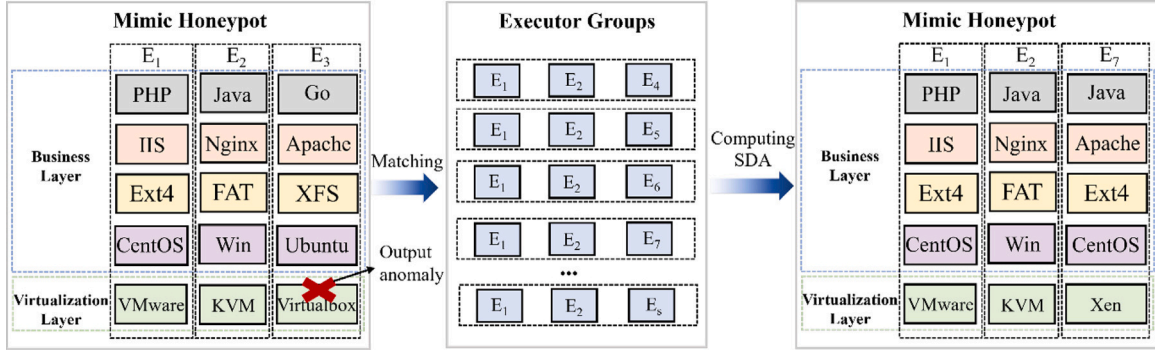


Fig. 3. Execution flow of the dynamic scheduling module.

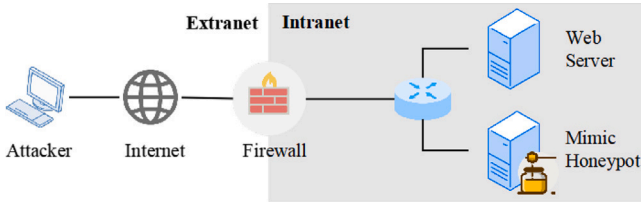


Fig. 4. Simulation network topology.

adjudication to select responses that contain more vulnerabilities to feed back to attackers, thereby attracting attackers to engage in deeper interactions with the honeypot.

In Section 3, we use the Minimax-Q algorithm to solve for the Bayesian Nash equilibrium of the mimic honeypot attack–defense model and obtain, for the current network state and attack situation, the executor group that yields the highest reward. The reward evaluation accounts for the honeypot’s deception capability, defensive capability, and resource overhead. Note that the Bayesian Nash equilibrium may not be unique; in such cases, one executor group that conforms to a Nash equilibrium outcome is chosen at random to compose the mimic honeypot.

When attack traffic arrives at the mimic honeypot, the business layer and virtualization layer of the executor groups produce results at different levels. We first apply majority adjudication to the virtualization-layer results. When an attacker attempts to attack the honeypot core (e.g., virtualization escape), the attack typically relies on specific vulnerabilities of a virtualization platform; a method that succeeds in bypassing one virtualization platform may have low probability of reproducing on another platform, and virtualization techniques that have not been escaped remain operable. If inconsistencies occur in the virtualization-layer data, there exists the possibility that the honeypot core has been compromised; in that case, executor groups whose virtualization outputs are anomalous should be replaced. A new executor group is invoked according to the dynamic scheduling policy to mask the abnormal perturbation of the affected executor group.

The dynamic scheduling module, given the current executor group of the mimic honeypot, filters executor groups to those that exclude any executor showing anomalies, then computes the security defense value (*SDV*) of each filtered executor group and selects the executor with the highest *SDV* for replacement. The execution flow of the dynamic scheduling module is shown in Fig. 3. Executor groups that pass the virtualization-layer adjudication are subsequently subjected to minority adjudication at the business layer; in this round of adjudication, results flagged as anomalous will be selected as the final output to further attract attackers to launch attacks against them.

5. Experiments and analysis

5.1. Simulation environment

To validate the proposed model and method, we built a simulated Web service system for experimental analysis, consisting of both external and internal networks, as illustrated in Fig. 4. The attacker originates from the external network, with the goal of breaching the system and obtaining sensitive information, while the internal network hosts both a Web server and a mimic honeypot server. Legitimate external users are only allowed to access the Web server, which in turn can communicate with the database server and the file server. The mimic honeypot is deployed within the internal network to emulate vulnerable Web services, thereby attracting attackers and enabling early detection of external attacks. The experimental hardware configuration consists of an AMD Ryzen 9 5900HS with Radeon Graphics (3.30 GHz), 8 cores/16 threads, 16.0 GB memory, running Windows 11 (version 26100.6584), 64-bit. The software environment is Python 3.9.

After setting up the experimental environment, we applied the vulnerability scanning tool Nuclei to analyze the servers and extract critical vulnerability information, as summarized in Table 1.

In the mimic honeypot attack–defense scenario, the defensive actions correspond to heterogeneous business executors. In this work, we constructed 50 such executors. Each executor group contains three executors, resulting in a total of C_3^5 executor groups. Since some defensive actions exhibit similar defensive effectiveness, clustering analysis was applied to group them, and the defense strategy set was defined accordingly.

In the attack–defense game scenario of mimic honeypots, defensive actions are constructed as heterogeneous business executors. Here, we build 50 types of business executors. Assuming that each executor group consists of 3 business executors, there are a total of C_3^{50} executor groups. The defensive action set is defined as $\{h_1, h_2, \dots, h_{C_3^{50}}\}$. Since the defensive effects of some actions are relatively similar, for the convenience of experimental study, we perform clustering analysis on the defensive actions. Based on the clustering results, the defensive strategy set is denoted as $H = \{H_1, H_2, \dots, H_7\}$.

- (1) Game participants $N = \{\text{attacker, honeypot}\}$ are the external attacker and the mimic honeypot.
- (2) Game state $S = \{s_1, s_2, \dots, s_6\}$, with specific meanings shown in Table 2.
- (3) Attacker’s action set $A = \{A_1, A_2, \dots, A_7\}$, where

$$\begin{aligned}
 A_1 &= \{\}, & A_2 &= \{a_1, a_6, a_{11}, a_{16}\}, \\
 A_3 &= \{a_2, a_7, a_{12}, a_{17}\}, \\
 A_4 &= \{a_3, a_5, a_8, a_9, a_{13}, a_{18}\}, \\
 A_5 &= \{a_4, a_9, a_{14}, a_{15}, a_{19}, a_{20}\}, \\
 A_6 &= \{a_1, a_6, a_{11}, a_{16}, a_{22}, a_{24}\}, \\
 A_7 &= \{a_4, a_9, a_{14}, a_{15}, a_{19}, a_{20}, a_{21}, a_{23}, a_{25}\}
 \end{aligned}$$

Table 1
Vulnerabilities information.

Layer	Type	ID	CVE	Service
Business	Script	a_1	CVE-2024-10516	php
		a_2	CVE-2023-39677	php
		a_3	CVE-2020-0618	C#
		a_4	CVE-2021-44228	java
		a_5	CVE-2024-23897	java
	Web server	a_6	CVE-2017-9805	Apache
		a_7	CVE-2023-46604	Apache
		a_8	CVE-2018-11759	Tomcat
		a_9	CVE-2020-9484	Tomcat
		a_{10}	CVE-2019-2725	WebLogic
	Database	a_{11}	CVE-2021-22145	ES
		a_{12}	CVE-2018-17246	ES
		a_{13}	CVE-2020-9483	MySQL
		a_{14}	CVE-2022-32022	MySQL
		a_{15}	CVE-2021-22205	Redis
	OS	a_{16}	CVE-2021-31324	CentOS
		a_{17}	CVE-2018-18323	CentOS
		a_{18}	CVE-2024-1061	CentOS
		a_{19}	CVE-2023-2640	Ubuntu
		a_{20}	CVE-2021-38146	Windows
Virtualization	Technology	a_{21}	CVE-2021-24364	KVM
		a_{22}	CVE-2024-10400	VirtualBox
		a_{23}	CVE-2022-21661	VirtualBox
		a_{24}	CVE-2021-22005	VMware
		a_{25}	CVE-2021-21972	VMware

Table 2
Network states.

State	Description
s_1	The attacker has not launched an attack; the system operates normally.
s_2	The attacker successfully compromises the real Web server.
s_3	The attacker fails to detect the mimic honeypot and continues to attack, engaging in deeper interactions.
s_4	The attacker detects the mimic honeypot and disconnects.
s_5	The attacker detects and successfully compromises the mimic honeypot itself.
s_6	The attacker detects but fails to compromise the mimic honeypot itself.

- (4) Mimic honeypot defense action set $H = \{H_1, H_2, \dots, H_7\}$.
- (5) Attacker types $\Phi = \{\varphi_1, \varphi_2\}$, where φ_1 denotes a high-capability attacker and φ_2 denotes a low-capability attacker.
- (6) Prior probability distribution of attacker types $P_A = \{p_A(s_1, \varphi_1), p_A(s_1, \varphi_2), p_A(s_2, \varphi_1), p_A(s_2, \varphi_2), p_A(s_3, \varphi_1), p_A(s_3, \varphi_2), p_A(s_4, \varphi_1), p_A(s_4, \varphi_2), p_A(s_5, \varphi_1), p_A(s_5, \varphi_2), p_A(s_6, \varphi_1), p_A(s_6, \varphi_2)\} = \{0.1, 0.9, 0.2, 0.8, 0.15, 0.85, 0.3, 0.7, 0.2, 0.8, 0.4, 0.6\}$.
- (7) In real networks, the state transition probabilities cannot be directly obtained. To verify whether the Bayesian Nash equilibrium solved by the Minimax-Q algorithm is correct, we set the state transition probabilities under a simulated experimental scenario. For example, when the state is s_3 , s_3 cannot directly transition to s_1 or s_2 , so the state transition probabilities are given as:

$$p(s_3) = \begin{bmatrix} 0.5 & 0.45 & 0.4 & \dots & 0.3 & 0.25 & 0.2 \\ 0.2 & 0.25 & 0.3 & \dots & 0.4 & 0.45 & 0.5 \\ 0.2 & 0.2 & 0.2 & \dots & 0.1 & 0.1 & 0.1 \\ 0.1 & 0.1 & 0.1 & \dots & 0.2 & 0.2 & 0.2 \end{bmatrix}. \quad (20)$$

Next, we will evaluate the effectiveness of the proposed method under the condition of unknown state transition probabilities in mimic honeypots.

Table 3
Parameter settings.

Set number	α	γ	ϵ
Set1	0.3	0.2	0.3
Set2	0.3	0.2	0.2
Set3	0.2	0.2	0.2
Set4	0.3	0.5	0.2

5.2. Results and analysis of the game

(1) Parameter Testing and Analysis

Parameters directly affect both the speed and effectiveness of the learning model, and the ability to converge to the correct Nash equilibrium within a reasonable time is critical for verifying the effectiveness of the algorithm. In Section 4.3, we conducted a preliminary analysis of the relevant parameters. Building on that, this section further tests different parameter values to identify suitable parameter settings for the attack–defense scenario. In our experiments, four different parameter configurations were tested, with the specific settings shown in Table 3.

Based on the above parameter settings, we tested the evolution results of the mimic honeypot defense strategy under state s_3 . As shown in Fig. 5, it can be observed that different parameter settings affect the learning speed and convergence of the algorithm. Under the parameter settings of Set2 and Set4, the algorithm converges relatively quickly, obtaining the optimal strategy within 150 iterations. A larger α (set to 0.3 here) indicates that the model can update the value function more rapidly and respond more sensitively to new information, thereby accelerating convergence. Under the Set4 configuration, the algorithm maintains relatively higher stability, which can be attributed to the higher γ (set to 0.5 here), emphasizing long-term rewards and thus facilitating the development of more robust defense strategies.

In addition, we evaluated the defense rewards of the mimic honeypot under different parameter settings. To ensure that the reward values do not merely reflect a single defense outcome, we computed the average rewards over 500 defense rounds. As shown in Fig. 6, the average reward of Set1 is significantly lower than that of the other settings. Furthermore, we calculated the average rewards over 15,000 defense rounds for the remaining three settings. The mean rewards of Set3 and Set4 are approximately 0.91, higher than that of Set2 (0.89). For a further comparison, we also calculated the standard deviation to reflect the dispersion of rewards. The standard deviation of Set4 is about 0.0105, lower than that of Set3 (0.0120). In summary, among the four parameter configurations, Set4 is the most suitable for this scenario. Since Set4 already achieves a satisfactory outcome that meets the experimental requirements, no further parameter optimization is performed. To validate the correctness of the proposed method, we employed the state transition probabilities defined in Section 5.2 to analyze the convergence of the game model under the Set4 configuration, as illustrated in Fig. 7. A comparison with Fig. 5(d) shows that the convergence results of the proposed method are fully consistent with those obtained using the known state transition probability matrix. It is worth noting, however, that the method based on the known state transition probability matrix requires approximately 200 iterations to converge and exhibits significant oscillations during the process, indicating relatively poor stability. In contrast, the proposed method not only converges accurately to the correct result but also significantly improves learning efficiency, thereby demonstrating clear advantages in accelerating convergence and optimizing defense strategies.

(2) Reward-Component Ablation and Sensitivity Analysis

To examine whether DV and SDV contain redundant information, we first calculated the Spearman rank correlation between the two components. The resulting correlation was negligible and statistically insignificant ($\rho_s = 0.0195$, $p = 0.2149$). This result indicates that DV and SDV exhibit no significant monotonic association and primarily characterize different evaluation dimensions.

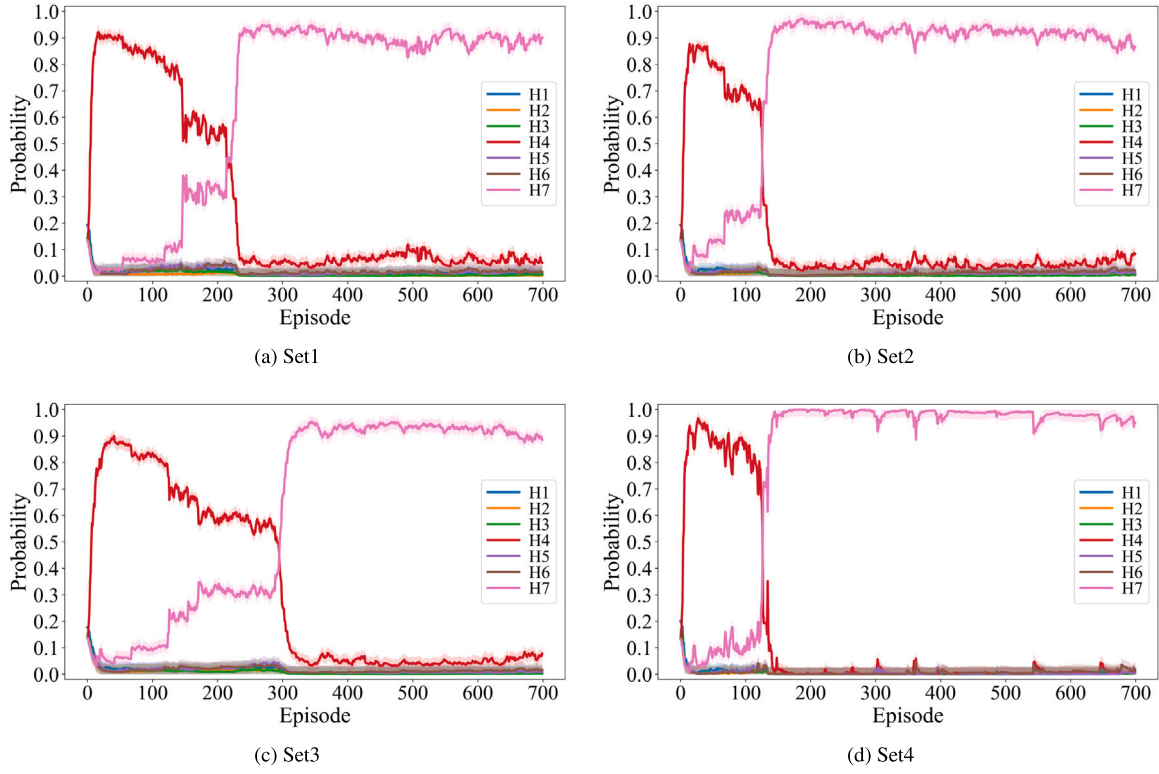


Fig. 5. Defensive decision-making under different parameter settings.

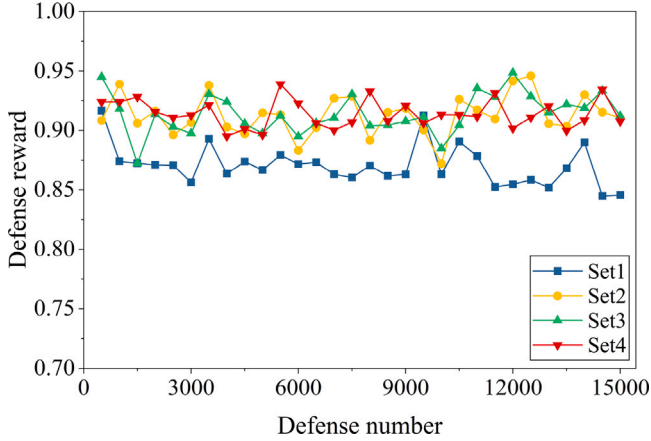


Fig. 6. Defense reward changes with different settings.

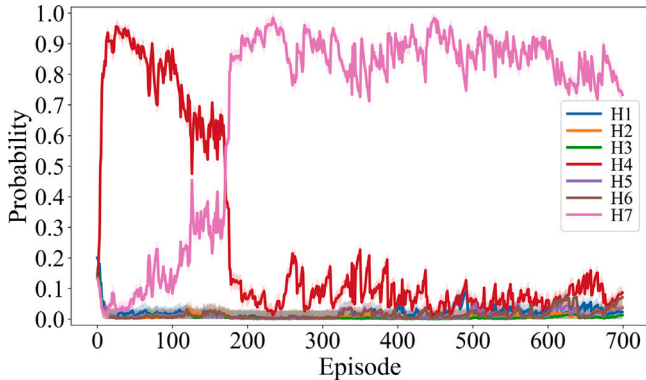


Fig. 7. Defensive decision-making under known state transition probabilities.

Table 4

Ablation results of the reward-function components.

Reward setting	Spearman rank correlation	Top-20 overlap
Full reward	1.0000	100%
Without <i>DV</i>	0.7056	25%
Without <i>SDV</i>	0.9094	70%
Without <i>Overhead</i>	0.9943	85%

We subsequently constructed three ablated reward functions by removing *DV*, *SDV* and *Overhead*, respectively. Because the reward function determines the relative preference among candidate defense actions, the resulting rankings were compared with the full model using the Spearman rank correlation and Top-20 overlap.

As shown in Table 4, removing *DV*, *SDV* and *Overhead* resulted in Spearman rank correlations of 0.7056, 0.9094, and 0.9943, and Top-20 overlaps of 25%, 70% and 85%, respectively. These results indicate that the three components play distinct and complementary roles in evaluating candidate defense actions.

To further evaluate the sensitivity of the reward function to the relative importance of its components, the reward function was generalized only for the sensitivity analysis as follows:

$$R_h(\alpha, \beta, \gamma) = \widetilde{DV}^\alpha * \widetilde{SDV}^\beta * \left(\frac{1}{\widetilde{Overhead}} \right)^\gamma, \quad (21)$$

where α , β , and γ denote the exponents of *DV*, *SDV* and *Overhead*, respectively. Normalized component values were used to eliminate differences in numerical scale. Each exponent was varied from 0 to 2 while the other two were fixed at 1.

As shown in Fig. 8, the candidate-action ranking remained generally stable under moderate perturbations around the unit-exponent setting $\alpha = \beta = \gamma = 1$, as indicated by the consistently high Spearman rank correlations and Top-20 overlaps. These results demonstrate that the ranking is robust to moderate variations in the component exponents

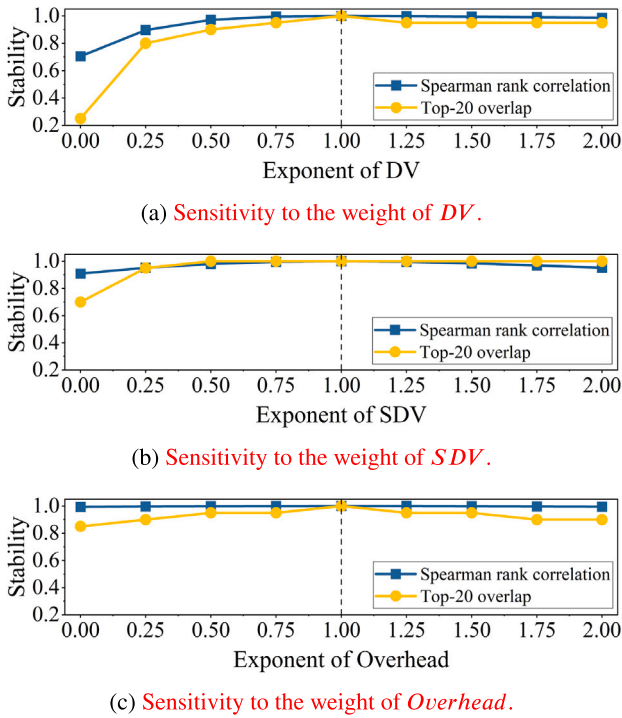


Fig. 8. Sensitivity analysis of reward-component exponents.

Table 5
Sensitivity analysis of the initial confidence.

Initial confidence	Dominant action	Dominant probability	Convergence rate
0.1	H_7	0.300	0%
0.3	H_7	0.737	30%
0.5	H_7	0.956	100%
0.7	H_7	0.989	100%
0.9	H_7	0.995	100%

and does not rely on a narrowly tuned parameter setting. The unit-exponent setting is therefore adopted as a neutral reference rather than as a mathematically optimal configuration.

(3) Initial-Confidence Sensitivity Analysis

To investigate the influence of the initial confidence setting on defense decision-making, five initial confidence values, namely 0.1, 0.3, 0.5, 0.7 and 0.9, were tested. For each setting, the Minimax-Q algorithm was run using 10 random seeds, with 700 learning episodes in each run, while all other experimental parameters were kept unchanged. The results of the initial confidence sensitivity analysis are presented in Table 5.

The ranking of candidate executor groups remained unchanged across all tested settings, with H_7 consistently identified as the dominant defense action. The initial confidence mainly affected strategy concentration and convergence: lower values produced more dispersed strategies and lower convergence rates, whereas values of 0.5 and above achieved convergence in all runs. Therefore, 0.5 was retained as a neutral setting that avoids imposing an excessively strong prior while ensuring stable learning performance.

(4) Comparative Testing and Analysis

Based on the parameter testing and analysis results above, Set4 is selected as the baseline. Our method is compared with several classical game-theoretic approaches (Yang et al., 2019; Huang et al., 2017; Lei et al., 2018) to observe the changes in defensive rewards under state s_3 . To further compare the proposed method with recent representative MARL/DRL approaches, we referred to the studies by Sun et al. (2025) and Xie et al. (2026). Specifically, the core learning mechanisms of

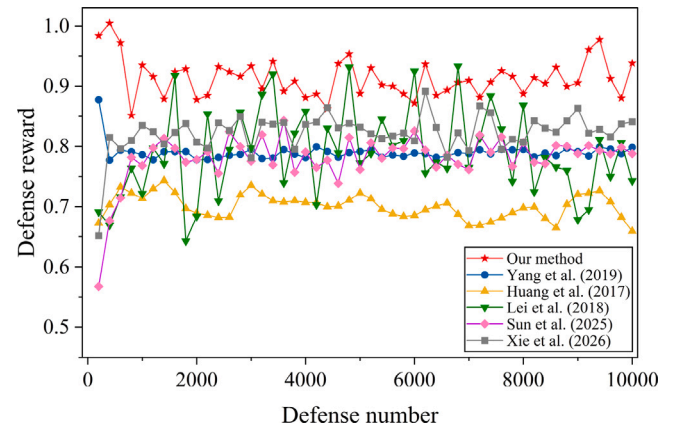


Fig. 9. Comparison of defense rewards of different methods.

WoLF-PHC and Nash Q-Network were adapted to the mimic-honeypot attack–defense environment constructed in this study and implemented as comparative baselines. To avoid reflecting only the outcome of a single game, the results are averaged over 200 iterations, which also provides a smoother and more interpretable curve, as shown in Fig. 9.

As shown in Fig. 9, the proposed method achieves and maintains the highest defensive reward across all iterations, converging to 0.9, and is significantly superior to the other five methods. The method in Yang et al. (2019) can gradually adjust strategies through learning during the game, and its reward curve is relatively stable overall. However, due to limitations in convergence speed and learning accuracy, the defensive reward remains around 0.78, which is noticeably lower than that of the proposed method. The approach in Huang et al. (2017), which is based on evolutionary game modeling, yields relatively fixed defensive strategies and fails to adequately respond to the dynamic changes in attacker strategies, resulting in lower overall rewards maintained within the range of 0.65–0.7. The method in Lei et al. (2018) adopts an incomplete-information Markov game to generate strategies, which in theory can describe asymmetric-information game scenarios. Nevertheless, because the state transition probabilities are difficult to estimate accurately, the reward curve fluctuates greatly, exhibiting instability at different stages of the iterations. The multi-agent WoLF-PHC method adopted by Sun et al. (2025) dynamically adjusts its learning rate according to the reward difference between the current policy and the average policy. Its defense reward remains at approximately 0.78. However, because it mainly relies on local policy hill-climbing updates, its overall performance is still lower than that of the proposed method. The Nash Q-Network proposed by Xie et al. (2026) uses deep neural networks to approximate value functions and learn Nash-equilibrium strategies, achieving an average defense reward of approximately 0.82. However, neural-network approximation and joint policy training may introduce certain estimation errors. By explicitly solving the minimax equilibrium and incorporating the prior probabilities of attacker types into strategy optimization, the proposed method achieves higher defense rewards and better stability in the discrete, zero-sum, and incomplete-information mimic-honeypot scenario constructed in this study.

Despite the above advantages, Minimax-Q has several limitations. Abrupt changes in the attacker strategy may make the learned Q-values temporarily inconsistent with the current environment, while simultaneous online updates of both players may cause strategy oscillations or slower convergence. Moreover, because the minimax strategy maximizes the guaranteed payoff against the worst-case opponent, it may be conservative when facing weak, irrational, or non-Nash attackers. The current study assumes a rational zero-sum game and predefined attacker-type priors. Therefore, the proposed method primarily focuses on guaranteeing defense performance under uncertain

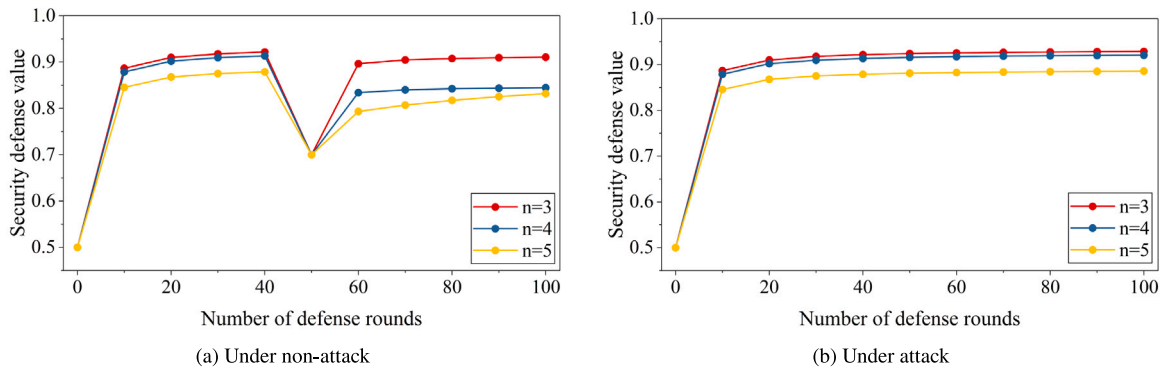


Fig. 10. Security defense value before and after attack.

attacks rather than exploiting non-equilibrium opponent behavior. Future work will consider opponent modeling, online prior estimation, and asynchronous updating mechanisms.

5.3. Security analysis

(1) Scheduling Effectiveness

The purpose of scheduling in the virtualization layer of the mimic honeypot is to replace abnormal virtualization technologies so that they do not affect the honeypot's outputs. If the scheduling module selects a new business executor whose virtualization layer is similar to the previous one, it may still contain platform vulnerabilities exploited by attackers. Therefore, we recorded the security defense value (SDV) before and after attacks to observe whether the SDV can return to a high level after the virtualization technology has been compromised, as shown in Fig. 10. This further explains the principle described in Section 3, where each mimic honeypot is configured with three business executors.

As shown in Fig. 10(a), under non-attack conditions, the proposed method maintains the SDV of the mimic honeypot at a high level. As the number of scheduling operations increases, the scheduling strategy proves effective even in the absence of attacks, gradually improving the credibility of the virtualization layer. Consequently, the SDV increases steadily based on the dynamic update mechanism and eventually stabilizes. When the number of executors is three, the SDV stabilizes at around 0.92. Increasing the number of executors raises the similarity of the virtualization layer, which in turn increases the likelihood of being detected by attackers, leading to a lower SDV .

As shown in Fig. 10(b), during the initial experimental phase when no attacks occur, the SDV increases steadily and stabilizes. Once an attack takes place, the SDV drops significantly, thereby reducing the credibility of the virtualization layer. In response, the dynamic scheduling module removes the abnormal business executors and replaces them with executors of the highest SDV among the currently normal executor group. As a result, the SDV gradually recovers and eventually stabilizes near its pre-attack level. This demonstrates that the scheduling strategy of the proposed mimic honeypot possesses strong adaptability, enabling rapid adjustment after virtualization layer attacks to ensure the overall security and stability of the system.

(2) Attack Success Rate

We use the probability of successful virtual escape attacks initiated by the attacker (i.e., attack success rate) to measure the security of the mimic honeypot. To evaluate its ability to resist backdoors from unknown vulnerabilities in the virtualization layer, we implanted virtual escape backdoors into the virtualization layer of each business executor and tested whether triggering these vulnerabilities could lead to successful escape. The proposed method is compared with a static honeypot, a dynamic honeypot (Dowling et al., 2020), a multi-layered honeypot (Shivaprasad et al., 2024), a MADDPG-based adaptive honeypot (Albaseer et al., 2024), and a risk-aware reinforcement learning

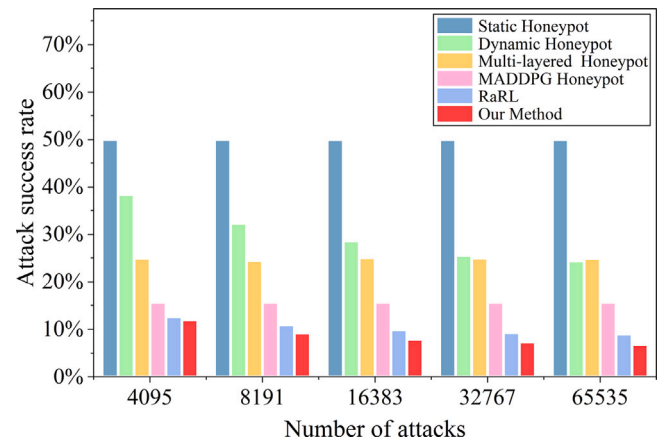


Fig. 11. Attack success rate.

scheme (RaRL) (Gao et al., 2024). The attack success rates under virtual escape attacks are shown in Fig. 11.

As shown in Fig. 11, across different test scales, the static honeypot is easily identified and exploited by attackers because of its fixed and rigid defense strategy. Its attack success rate remains at approximately 50%, indicating relatively weak defensive capability. By contrast, the dynamic honeypot introduces a certain degree of uncertainty through periodic strategy adjustments. However, its strong regularity makes its behavior easier for attackers to predict, resulting in an attack success rate of approximately 30%–40%. The multi-layered honeypot increases the attack threshold through structural complexity and reduces the attack success rate to approximately 25%, although the overall improvement remains limited. The MADDPG-based adaptive honeypot uses multi-agent deep reinforcement learning to dynamically optimize honeypot deployment according to environmental changes, providing favorable adaptability and distributed decision-making capability. However, it lacks a dedicated mechanism for defending against virtual escape attacks, and its attack success rate therefore remains at approximately 15%. RaRL introduces risk constraints into the reinforcement learning process to avoid selecting high-risk defense actions, further reducing the attack success rate to approximately 8%–13%. Nevertheless, it mainly optimizes the trade-off between system risk and defense utility, and its scalability is limited when the number of servers changes. Compared with the above methods, the proposed mimic honeypot adaptive defense generation method exhibits significant advantages against virtual escape attacks, with attack success rates consistently below 12% and further reduced to approximately 6%–8% in large-scale experiments. This performance benefits from its game-theoretic adaptive defense mechanism, which promptly isolates and removes abnormal executors upon detection and discards related

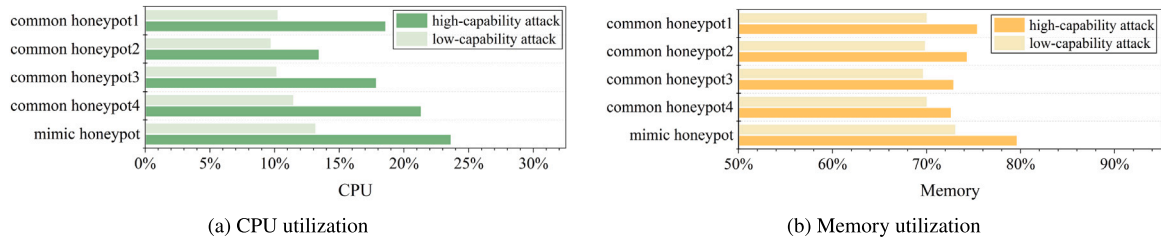


Fig. 12. Comparison of performance overhead.

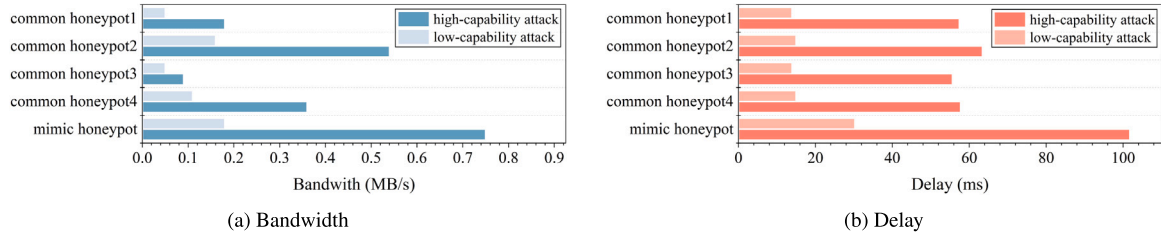


Fig. 13. Comparison of network load.

feedback, thereby eliminating the possibility of attackers repeatedly exploiting the same vulnerability or backdoor. Such a dynamic and flexible defense mechanism enables the mimic honeypot to effectively suppress virtual escape attacks and substantially enhance the system's security and stability.

5.4. Overhead analysis

(1) Performance Overhead

The mimic honeypot adopts dynamic heterogeneous redundancy and game-theoretic decision-making, which inevitably introduces additional CPU and memory overhead. Excessive overhead could restrict deployment scope and diminish system practicality. Therefore, we measured and compared CPU and memory usage of the mimic honeypot and a conventional virtual honeypot within a single attack–defense cycle.

As shown in Fig. 12, the mimic honeypot consistently exhibits higher resource consumption under both low-capability and high-capability attackers. CPU utilization increases by approximately 2%–9%, while memory usage rises by 3%–7%. These results align with expectations: although the extra overhead is noticeable, it remains within an acceptable range and does not significantly affect the deployability or scalability of the system.

(2) Network Load

During operation, the mimic honeypot must dynamically schedule multiple heterogeneous executors based on game results, impacting bandwidth usage and communication latency—two indicators closely tied to honeypot effectiveness and stealth. Excessive network load may delay strategy switching or raise suspicion from attackers. Thus, maintaining network performance close to that of conventional honeypots is essential.

As illustrated in Fig. 13, when defending against high-capability attackers, the mimic honeypot triggers additional data exchanges, resulting in bandwidth usage of about 0.75 MB/s, compared to below 0.54 MB/s for conventional honeypots. Communication delay shows the same pattern: approximately 100 ms for the mimic honeypot versus around 60 ms for conventional honeypots. This reflects the cost of sustaining robust defensive actions under stronger attacks. In contrast, against low-capability attackers, the mimic honeypot introduces only minor additional overhead, with bandwidth and latency remaining close to baseline levels. This demonstrates its ability to adaptively adjust defense intensity and balance performance with security.

(3) Computational Latency

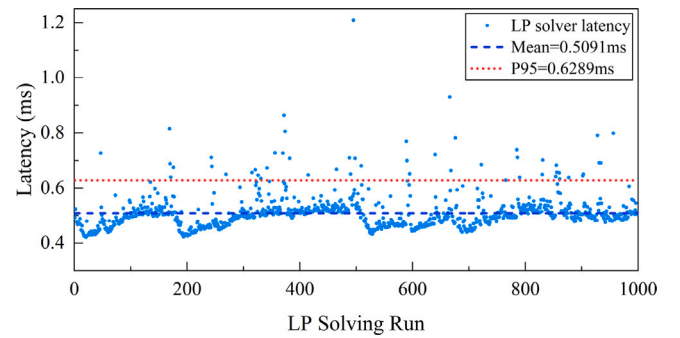


Fig. 14. LP-solving Latency.

The defense strategy at each decision step is obtained by solving a linear programming problem. To evaluate the resulting computational overhead, 100 warm-up runs were first performed, followed by 1000 independent LP-solving runs on the same experimental platform.

As shown in Fig. 14, the 95th-percentile solving latency was 0.629 ms, indicating that 95% of the LP problems were solved within 0.629 ms. This latency is substantially lower than the approximately 100 ms communication latency observed under high-capability attacks, demonstrating that LP solving is not the primary system latency bottleneck under the current defense action space and introduces only limited overhead into online defense decision-making. It should be noted that increasing the numbers of attack and defense actions or attacker types will enlarge the number of decision variables and constraints, potentially increasing the solving latency. Therefore, future work will further evaluate the computational performance and scalability of the proposed method in larger attack–defense action spaces.

Overall, the mimic honeypot incurs higher overhead in CPU, memory, bandwidth, and latency—especially under high-capability attacks. However, its dynamic game-theoretic and mimicry-based defense mechanisms substantially improve its resistance to virtual escape and reduce attack success rates. In high-security environments, these additional costs are reasonable and reflect an appropriate trade-off between system performance and enhanced security.

6. Conclusion

To address the challenges of enhancing the deception capability and intrinsic security of virtual honeypots and strengthening the defender's

dominance in asymmetric attack–defense scenarios, this paper proposes an adaptive defense generation method for mimic honeypots based on Minimax-Q game theory. The method formulates dynamic games between the attacker and the honeypot, evaluates business executors by jointly considering deception value, security defense value, and computational overhead, and employs a multi-agent reinforcement learning-based Minimax-Q algorithm to solve for the Bayesian Nash equilibrium, thereby enabling the adaptive generation of dynamic and flexible honeypots to cope with complex attack scenarios. Experimental results demonstrate that the proposed mimic honeypot not only improves the capture of attack behaviors but also enhances its intrinsic security. Even when attackers identify and launch targeted attacks, the dynamic adjustment mechanism of business executors ensures effective defense and keeps potential threats under control. While the present study primarily validates effectiveness in a simulation environment, future work will focus on deploying mimic honeypots in real-world networks to comprehensively assess their practical performance and applicability.

CRedit authorship contribution statement

Shangdong Liu: Writing – review & editing, Methodology, Conceptualization. **Xukun Qian:** Writing – original draft, Formal analysis, Data curation. **Yingrui Cai:** Visualization, Software. **Sisi Shao:** Supervision, Methodology. **Wenbo Wang:** Supervision. **Fei Wu:** Writing – review & editing. **Yimu Ji:** Resources, Funding acquisition.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This work was funded by the National Key R&D Program of China (2023YFB2904004, 2023YFB2904000), Jiangsu Key Development Planning Project (BE2023004-2), and the Open Research Fund of The State Key Laboratory of Blockchain and Data Security, Zhejiang University.

Data availability

Data will be made available on request.

References

- Albaseer, A., Hamood, M., Abdallah, M., 2024. Adaptive honeypot defense deployment: A stackelberg game approach with decentralized DRL for AMI protection. In: 2024 International Conference on Computing, Networking and Communications. ICNC, pp. 1–5. <http://dx.doi.org/10.1109/ICNC59896.2024.10556285>.
- Anwar, A.H., Kamhoua, C.A., Leslie, N., Kiekintveld, C.D., 2021. Honeypot allocation games over attack graphs for cyber deception. In: Kamhoua, C.A., Kiekintveld, C.D., Fang, F., Zhu, Q. (Eds.), *Game Theory and Machine Learning for Cyber Security*. John Wiley & Sons, Ltd, pp. 62–76. <http://dx.doi.org/10.1002/9781119723950.ch4>.
- Anwar, A.H., Kamhoua, C.A., Leslie, N.O., Kiekintveld, C., 2022. Honeypot allocation for cyber deception under uncertainty. *IEEE Trans. Netw. Serv. Manag.* 19 (3), 3438–3452. <http://dx.doi.org/10.1109/TNSM.2022.3179965>.
- Aslan, Ö., Aktuğ, S.S., Ozkan-Okay, M., Yilmaz, A.A., Akin, E., 2023. A comprehensive review of cyber security vulnerabilities, threats, attacks, and solutions. *Electronics* 12 (6), 1333. <http://dx.doi.org/10.3390/electronics12061333>.
- Chen, Z., Cui, G., Zhang, L., Yang, X., Li, H., Zhao, Y., Ma, C., Sun, T., 2021. Optimal strategy for cyberspace mimic defense based on game theory. *IEEE Access* 9, 68376–68386. <http://dx.doi.org/10.1109/ACCESS.2021.3077075>.
- Chernikova, A., Gozzi, N., Perra, N., Boboila, S., Eliassi-Rad, T., Oprea, A., 2023. Modeling self-propagating malware with epidemiological models. *Appl. Netw. Sci.* 8, 52. <http://dx.doi.org/10.1007/s41109-023-00578-z>.

- Commeey, D., Nkoom, M., Hounsinnou, S.G., Crosby, G.V., 2025. Blockchain-enabled dynamic honeypot conversion for resource-efficient IoT security. *J. Inf. Secur. Appl.* 93, 104109. <http://dx.doi.org/10.1016/j.jisa.2025.104109>.
- Dowling, S., Schukat, M., Barrett, E., 2020. New framework for adaptive and agile honeypots. *ETRI J.* 42 (6), 965–975. <http://dx.doi.org/10.4218/etrij.2019-0155>.
- Franco, J., Aris, A., Canberk, B., Uluagac, A.S., 2021. A survey of honeypots and honeynets for internet of things, industrial internet of things, and cyber-physical systems. *IEEE Commun. Surv. Tutor.* 23 (4), 2351–2383. <http://dx.doi.org/10.1109/COMST.2021.3106669>.
- Gao, D., Guo, C., Zhang, Y., Ji, W., Lv, Z., Li, Z., Zhang, K., Lin, R., 2024. Risk-aware SDN defense framework against anti-honeypot attacks using safe reinforcement learning. *Int. J. Netw. Manag.* 34 (6), <http://dx.doi.org/10.1002/nem.2297>.
- Geng, J., Wang, J., Fang, Z., Zhou, Y., Wu, D., Ge, W., 2024. A survey of strategy-driven evasion methods for PE malware: Transformation, concealment, and attack. *Comput. Secur.* 137, 103595. <http://dx.doi.org/10.1016/j.cose.2023.103595>.
- He, W., Tan, J., Guo, Y., Shang, K., Zhang, H., 2024. A deep reinforcement learning-based deception asset selection algorithm in differential games. *IEEE Trans. Inf. Forensics Secur.* 19, 8353–8368. <http://dx.doi.org/10.1109/TIFS.2024.3451189>.
- Hu, H., Wu, J., Wang, Z., Cheng, G., 2018. Mimic defense: a designed-in cybersecurity defense framework. *IET Inf. Secur.* 12 (3), 226–237. <http://dx.doi.org/10.1049/iet-ifs.2017.0086>.
- Huang, J., Zhang, H., Wang, J., Huang, S., 2017. Defense strategies selection based on attack-defense evolutionary game model. *J. Commun.* 38 (1), 168–176. <http://www.jocnline.com.cn/en/article/doi/10.11959/j.issn.1000-436x.2017019/>.
- Ilg, N., Duplys, P., Sisejkovic, D., Menth, M., 2023. A survey of contemporary open-source honeypots, frameworks, and tools. *J. Netw. Comput. Appl.* 220, 103737. <http://dx.doi.org/10.1016/j.jnca.2023.103737>.
- Javadpour, A., Jafari, F., Taleb, T., Shojafar, M., Benzaid, C., 2024. A comprehensive survey on cyber deception techniques to improve honeypot performance. *Comput. Secur.* 140, 103792. <http://dx.doi.org/10.1016/j.cose.2024.103792>.
- Kelly, C., Pitropakis, N., Mylonas, A., McKeown, S., Buchanan, W.J., 2021. A comparative analysis of honeypots on different cloud platforms. *Sensors* 21 (7), 2433. <http://dx.doi.org/10.3390/s21072433>.
- Lei, C., Zhang, H.-Q., Wan, L.-M., Liu, L., Ma, D.-h., 2018. Incomplete information Markov game theoretic approach to strategy generation for moving target defense. *Comput. Commun.* 116, 184–199. <http://dx.doi.org/10.1016/j.comcom.2017.12.001>.
- Li, D., Tian, S., Jin, W., Peng, J., Duan, M., 2025. Towards a moving target defense based on stochastic games and honeypots. *Inform. Sci.* 720, 122488. <http://dx.doi.org/10.1016/j.ins.2025.122488>.
- Liu, X., Wang, H., Li, C., 2024. A review of endogenous security research. *Electronics* 13 (11), 2185. <http://dx.doi.org/10.3390/electronics13112185>.
- Mohammadzad, M., Karimpour, J., 2023. Using rootkits hiding techniques to conceal honeypot functionality. *J. Netw. Comput. Appl.* 214, 103606. <http://dx.doi.org/10.1016/j.jnca.2023.103606>.
- Naik, N., Shang, C., Jenkins, P., Shen, Q., 2021. D-FRI-Honeypot: A secure sting operation for hacking the hackers using dynamic fuzzy rule interpolation. *IEEE Trans. Emerg. Top. Comput. Intell.* 5 (6), 893–907. <http://dx.doi.org/10.1109/TETCI.2020.3023447>.
- Panda, S., Rass, S., Moschyiannis, S., Liang, K., Loukas, G., Panaousis, E., 2022. HoneyCar: a framework to configure honeypot vulnerabilities on the internet of vehicles. *IEEE Access* 10, 104671–104685. <http://dx.doi.org/10.1109/ACCESS.2022.3210117>.
- Pawllick, J., Colbert, E., Zhu, Q., 2019. Modeling and analysis of leaky deception using signaling games with evidence. *IEEE Trans. Inf. Forensics Secur.* 14 (7), 1871–1886. <http://dx.doi.org/10.1109/TIFS.2018.2886472>.
- Perwej, Y., Abbas, S.Q., Dixit, J.P., Akhtar, N., Jaiswal, A.K., 2021. A systematic literature review on the cyber security. *Int. J. Sci. Res. Manag.* 9 (12), 669–710. <http://dx.doi.org/10.18535/ijrm/v9i12.ec04>.
- Rehman, Z., Gondal, I., Ge, M., Dong, H., Gregory, M., Tari, Z., 2024. Proactive defense mechanism: Enhancing IoT security through diversity-based moving target defense and cyber deception. *Comput. Secur.* 139, 103685. <http://dx.doi.org/10.1016/j.cose.2023.103685>.
- Safaei Pour, M., Nader, C., Friday, K., Bou-Harb, E., 2023. A comprehensive survey of recent internet measurement techniques for cyber security. *Comput. Secur.* 128, 103123. <http://dx.doi.org/10.1016/j.cose.2023.103123>.
- Sayed, M.A., Anwar, A.H., Kiekintveld, C., Kamhoua, C., 2023. Honeypot allocation for cyber deception in dynamic tactical networks: A game theoretic approach. In: Fu, J., Kroupa, T., Hayel, Y. (Eds.), *Decision and Game Theory for Security*. Springer Nature Switzerland, Cham, pp. 195–214. http://dx.doi.org/10.1007/978-3-031-50670-3_10.
- Selva Kumar, V., Mohan Raj, K., Gopalakrishnan, S., Vennila, G., Dhinakaran, D., Kavitha, P., 2025. Adaptive distributed honeypot detection network for enhanced cybersecurity against DoS and DDoS attacks. *Results Eng.* 26, 105521. <http://dx.doi.org/10.1016/j.rineng.2025.105521>.
- Shao, S., Ji, Z., Ji, Y., Datta, A., 2026. A game theory-based method for secure deployment of mimic honeynet. *IEEE Trans. Dependable Secur. Comput.* 23 (1), 799–813. <http://dx.doi.org/10.1109/TDSC.2025.3611450>.

- Shao, S., Ji, Z., Qian, X., Wu, F., Liu, S., Ji, Y., 2025. A mimic honeypot construction method based on incomplete information zero-sum stochastic games and Q-learning. *IEEE Internet Things J.* 12 (23), 50262–50278. <http://dx.doi.org/10.1109/JIOT.2025.3609800>.
- Shao, S., Ji, Y., Zhang, W., Liu, S., Jiang, F., Cao, Z., Wu, F., Zeng, F., Zuo, J., Zhou, L., 2023. A DHR executor selection algorithm based on historical credibility and dissimilarity clustering. *Sci. China Inf. Sci.* 66 (11), 212304. <http://dx.doi.org/10.1007/s11432-022-3635-2>.
- Shivaprasad, T., Moulyya, A., Guruprasad, N., 2024. Enhancing network security through a multi-layered honeypot architecture with integrated network monitoring tools. In: 2024 11th International Conference on Computing for Sustainable Global Development. *INDIACom, IEEE*, pp. 473–477. <http://dx.doi.org/10.23919/INDIACom61295.2024.10498895>.
- Siwakoti, Y.R., Bhurtel, M., Rawat, D.B., Oest, A., Johnson, R.C., 2023. Advances in IoT security: Vulnerabilities, enabled criminal services, attacks, and countermeasures. *IEEE Internet Things J.* 10 (13), 11224–11239. <http://dx.doi.org/10.1109/JIOT.2023.3252594>.
- Sun, R., Fei, J., Zhu, Y., Guo, Z., 2025. Multi-agent reinforcement learning for moving target defense temporal decision-making approach based on stackelberg-flipit games. *Comput. Mater. Contin.* 84 (2), 3765–3786. <http://dx.doi.org/10.32604/cmc.2025.064849>.
- Tian, W., Du, M., Ji, X., Liu, G., Dai, Y., Han, Z., 2021. Honeypot detection strategy against advanced persistent threats in industrial internet of things: A prospect theoretic game. *IEEE Internet Things J.* 8 (24), 17372–17381. <http://dx.doi.org/10.1109/JIOT.2021.3080527>.
- Touré, A., Imine, Y., Semnont, A., Delot, T., Gallais, A., 2024. A framework for detecting zero-day exploits in network flows. *Comput. Netw.* 248, 110476. <http://dx.doi.org/10.1016/j.comnet.2024.110476>.
- Veluchamy, S., Kathavarayan, R.S., 2022. Deep reinforcement learning for building honeypots against runtime DoS attack. *Int. J. Intell. Syst.* 37 (7), 3981–4007. <http://dx.doi.org/10.1002/int.22708>.
- Wang, X., Shi, L., Cao, C., Wu, W., Zhao, Z., Wang, Y., Wang, K., 2024. Game analysis and decision making optimization of evolutionary dynamic honeypot. *Comput. Electr. Eng.* 119, 109534. <http://dx.doi.org/10.1016/j.compeleceng.2024.109534>.
- Wu, J., 2022. Cyberspace endogenous safety and security. *Engineering* 15, 179–185. <http://dx.doi.org/10.1016/j.eng.2021.05.015>.
- Wu, Y., Jiang, N., Pham, H.V., Lutellier, T., Davis, J., Tan, L., Babkin, P., Shah, S., 2023. How effective are neural networks for fixing security vulnerabilities. In: *Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis*. In: *ISSTA 2023, Association for Computing Machinery*, New York, NY, USA, pp. 1282–1294. <http://dx.doi.org/10.1145/3597926.3598135>.
- Xie, Q., Koh, E., Cadet, X., Chin, P., 2026. Nash Q-network for multi-agent cybersecurity simulation. In: *Baras, J.S., Papavassiliou, S., Tsiropoulou, E.E., Sayin, M.O. (Eds.), Game Theory and AI for Security*. Springer Nature Switzerland, Cham, pp. 43–60. http://dx.doi.org/10.1007/978-3-032-08067-7_3.
- Yang, J., Zhang, H., Zhang, C., 2019. Network defense decision-making method based on stochastic game and improved WoLF-PHC. *J. Comput. Res. Dev.* 56 (5), 942–954. <http://dx.doi.org/10.7544/issn1000-1239.2019.20180877>.
- Zambianco, M., Facchinetti, C., Doriguzzi-Corin, R., Siracusa, D., 2024. Resource-aware cyber deception for microservice-based applications. *IEEE Trans. Serv. Comput.* 17 (6), 4211–4224. <http://dx.doi.org/10.1109/TSC.2024.3395919>.
- Zarca, A.M., Bernabe, J.B., Skarmeta, A., Alcaraz Calero, J.M., 2020. Virtual IoT HoneyNets to mitigate cyberattacks in SDN/NFV-Enabled IoT networks. *IEEE J. Sel. Areas Commun.* 38 (6), 1262–1277. <http://dx.doi.org/10.1109/JSAC.2020.2986621>.
- Zheng, Y., Li, Z., Xu, X., Zhao, Q., 2022. Dynamic defenses in cyber security: Techniques, methods and challenges. *Digit. Commun. Netw.* 8 (4), 422–435. <http://dx.doi.org/10.1016/j.dcan.2021.07.006>.
- Zhu, H., Liu, M., Chen, B., Che, X., Cheng, P., Deng, R., 2024. HoneyJudge: A PLC honeypot identification framework based on device memory testing. *IEEE Trans. Inf. Forensics Secur.* 19, 6028–6043. <http://dx.doi.org/10.1109/TIFS.2024.3407520>.



Shangdong Liu is an Associate Professor at the School of Computer Science, Software College and School of Cyberspace Security in Nanjing University of Posts and Telecommunications, as well as a Master's Supervisor. He holds a Ph.D. in Computer Architecture from Southeast University. He is a Senior Member of the China Computer Federation (CCF), a Member of the China Institute of Communications, a Member of the ACM, and a Member of the IEEE. He has published over 60 high-quality papers, obtained 8 invention patents, and secured 4 software copyrights. Additionally, he has contributed to one academic monograph and co-authored two textbooks. His research interests include cyberspace security, big data and cloud computing, and artificial intelligence.



Xukun Qian was born in 2000, received the B.E. degree in Information Security from Nanjing University of Posts and Telecommunications (NJUPT) in 2023. She is currently working towards the master degree in School of Computer Science, NJUPT. Her main research interests include endogenous safety, cyberspace security and mimic defense.



Yingrui Cai was born in 2004, received the B.S. degree in Computer Science and Technology from Nanjing Audit University in 2025. He is currently working towards the master degree in Nanjing University of Posts and Telecommunications. His current research is endogenous security of cyberspace security and mimic defense.



Sisi Shao was born in 1997, received the Ph.D. degree in Nanjing University of Posts and Telecommunications (NJUPT) in 2025. She is currently a Lecturer in School of Cyber Science and Engineering, Wuxi University. Her main research interests include cyberspace security, endogenous safety and mimic defense.



Wenbo Wang received the M.S. degree in Computer Science and Technology from Nanjing University of Posts and Telecommunications (NJUPT) in 2025. He is currently pursuing the Ph.D. degree in Computer Science and Technology at the same university. His main research interests include cybersecurity and machine learning.



Fei Wu received the Ph.D. degree in Information and Communication Engineering from Nanjing University of Posts and Telecommunications (NJUPT), China, in 2016. He is currently an associate professor with the College of Automation in NJUPT. He has authored over eighty scientific papers including TPAMI, TIP and TSE. His research interests include pattern recognition, artificial intelligence, and computer vision.



Yimu Ji was born in 1978, received the Ph.D. degree in Nanjing University of Posts and Telecommunications (NJUPT) in 2007. He is currently an Professor in School of Computer Science, NJUPT. His main research interests include the security and applications in cloud computing, big data, IoT and AI. He presided over and participated in many national, provincial and ministerial projects, obtained 7 provincial and municipal science and technology progress awards, published and employed more than 20 SCI papers, and obtained 4 authorized invention patents and 3 software copyrights. Among them, from 2009 to 2011, he served as a temporary deputy director of the Administrative Committee of Wujin National High tech Zone in Jiangsu Province (assisting in the work of scientific and technological talents, and won the title of preferred communist party member during this period), and from September to December 2012, he was sent to the University of Virginia for a UVA study visit, and from June to December 2016, he was sent to Purdue University for a study visit by the Jiangsu Provincial Government.